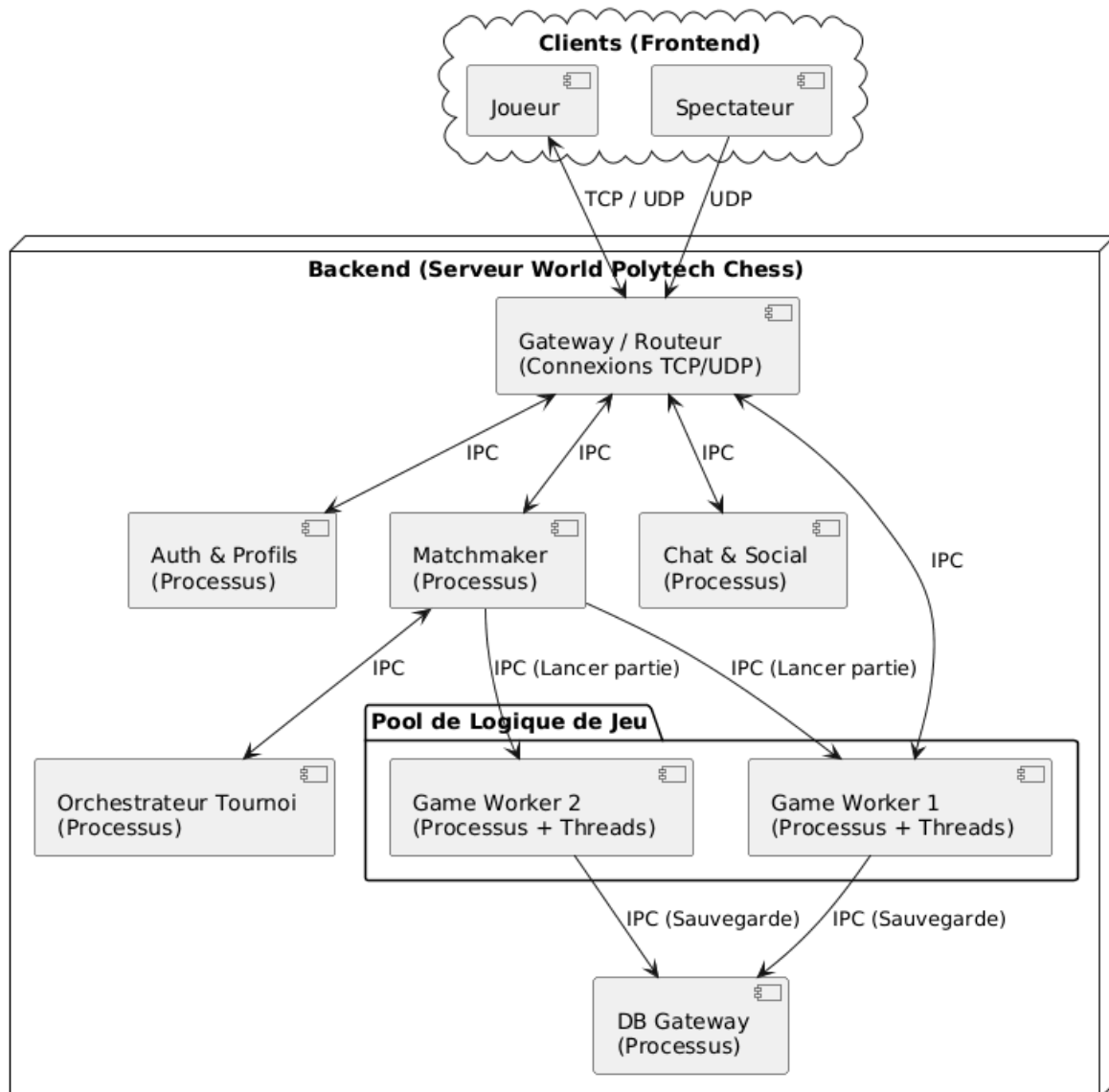




1. Pourquoi séparer les tâches ?

Architecture Modulaire => On sépare les tâches : un processus pour chaque fonctionnalités

2. Pourquoi des Processus ET des Threads ?

Processus pour les modules de haut niveau :

Les processus ont des espaces mémoire séparés. Si un module a une fuite de mémoire et crash, le système d'exploitation le tue, mais les "Game Workers" qui hébergent les parties en cours ne s'en rendent même pas compte.

Les Threads à l'intérieur des Game Workers :

Créer un processus est lourd pour le processeur : on ne peut pas lancer trop de parties en même temps sans faire crash le serveur. Les threads partagent la même mémoire, sont très légers à créer et permettent de gérer des centaines de parties simultanément au sein d'un même processus "Game Worker" avec une latence minimale.

3. Pourquoi l'IPC (Inter-Process Communication) et lesquels choisir ?

Puisque on sépare nos tâches en plusieurs modules, ces derniers sont isolés. Ils ne peuvent pas se passer une variable directement. Il faut construire des "ponts" entre eux : l'IPC.

- **Choix 1 : Les Files de Messages (Message Queues / POSIX Queues ou Sockets Unix)**
 - Où l'utiliser ? Entre le Gateway et le Matchmaker, ou entre le Matchmaker et les Game Workers.
 - Pourquoi ? C'est asynchrone. Si 1000 joueurs demandent une partie en même temps, le Gateway dépose 1000 messages dans la file. Le Matchmaker les dépile à son rythme, sans être submergé et sans bloquer le reste du serveur.
- **Choix 2 : La Mémoire Partagée (Shared Memory)**
 - Où l'utiliser ? Pour l'état global du Tournoi Orchestrator (ex: l'arbre du tournoi).
 - Pourquoi ? C'est le mécanisme d'IPC le plus rapide. Le module de Tournoi met à jour l'arbre en mémoire partagée, et le Gateway peut le lire instantanément pour l'envoyer aux spectateurs, sans avoir à s'envoyer des milliers de messages.

4. Pourquoi TCP et UDP pour communiquer avec les joueurs ?

- **TCP**
 - TCP garantit que les données arrivent dans le bon ordre et sans perte (il renvoie le paquet s'il est perdu).
 - Pour : L'authentification, le Chat, et surtout les Coups d'échecs. Les coups joués par les joueurs sont critiques. Si le paquet se perd sur le réseau, la partie est désynchronisée. TCP garantit que le coup arrivera au serveur.
- **UDP**
 - UDP envoie les données rapidement, sans vérifier si elles sont bien arrivées. S'il y a une perte, il ne s'arrête pas, il continue d'envoyer la suite.
 - Pour : Les spectateurs : Si 5000 spectateurs regardent un match, le serveur leur envoie l'état de l'échiquier et le chronomètre 10 fois par seconde en UDP. Si un spectateur a une micro-coupure réseau et perd un paquet, ce n'est pas grave : le paquet suivant arrivera 100 millisecondes plus tard avec l'image à jour. Utiliser TCP ici surcharge le serveur avec des demandes de renvoi inutiles.

Architecture du projet (idées) :

```
world_chess_tournament/
|
|— shared/          # Le code commun partagé par tous les processus
|
| |— jeu de Elliot/  # Moteur d'échecs (validation des coups, échec et mat)
|
| | |— ipc_utils/    # Fonctions pour lire/écrire dans les files de messages ou la mémoire partagée
|
| |— network_models/ # Structures des paquets TCP/UDP (ex: struct PlayerMove)
|
|
|— cmd/             # Les points d'entrée (les processus)
|
| |— gateway/
|
| | |— main.c        # Lance le routeur TCP/UDP
|
| | |— connection.c  # Gère les sockets clients
|
| |
|
| |— matchmaker/
|
| | |— main.c        # Lance la boucle de matchmaking
|
| | |— queue.c       # Logique des files d'attente par Elo
|
| |
|
| |— game_worker/
|
| | |— main.c        # Initialise le processus
|
| | |— room_manager.c # Gère les threads pour chaque salon actif
|
| | |— game_thread.c  # La boucle d'une partie spécifique
|
| |
|
| |— ... (pareil pour auth/, chat/, tournament/)
|
|
|— Makefile (ou CMakeLists.txt) # Pour tout compiler
```